BITCOIN MINING ACCELERATOR

OVERVIEW

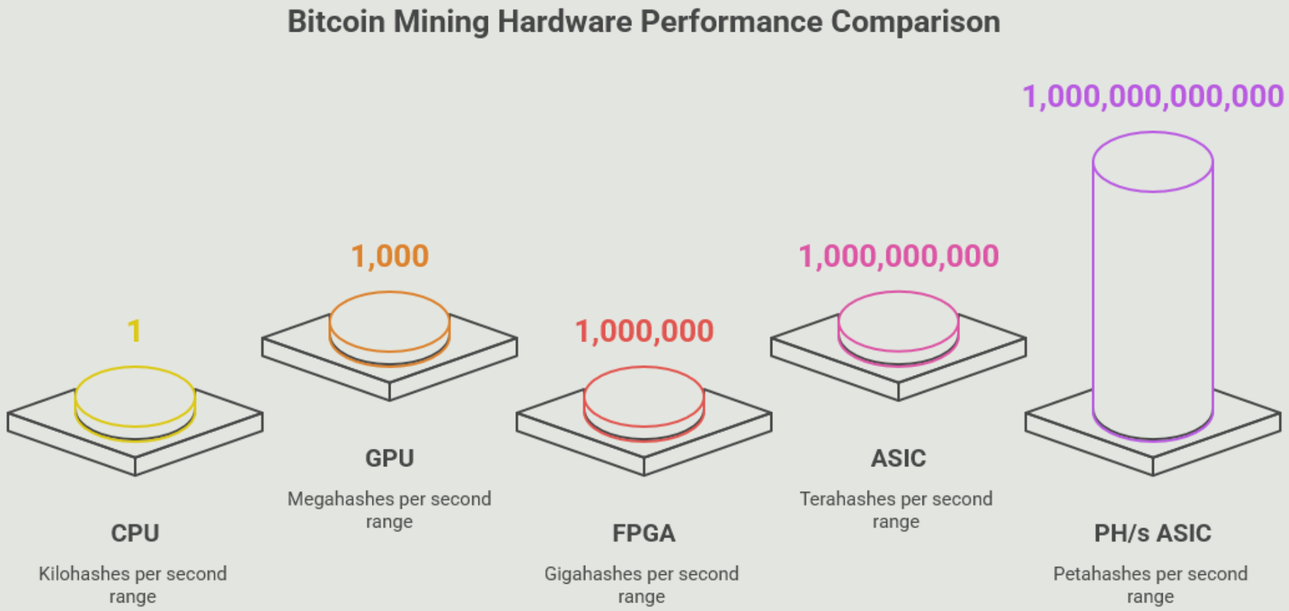
- Abstract
- Introduction
- Challenges
- Working Principle
- Working
- Reference Work
- Conclusion

ABSTRACT

- **Goal**: To design and develop a specialized hardware accelerator specifically for Bitcoin mining operations.
- **Core Challenge**: Bitcoin mining relies heavily on the repeated and computationally intensive calculation of the SHA-256 double-hash (SHA256d) algorithm, which is inefficient on general-purpose hardware.
- **Initial Focus**: This phase concentrates on implementing a fully functional and correct SHA256d computation unit directly in hardware using Verilog.
- **Benefit**: This foundational work demonstrates the inherent advantages of custom hardware in terms of speed and efficiency for this specific cryptographic workload, laying the groundwork for more advanced designs.

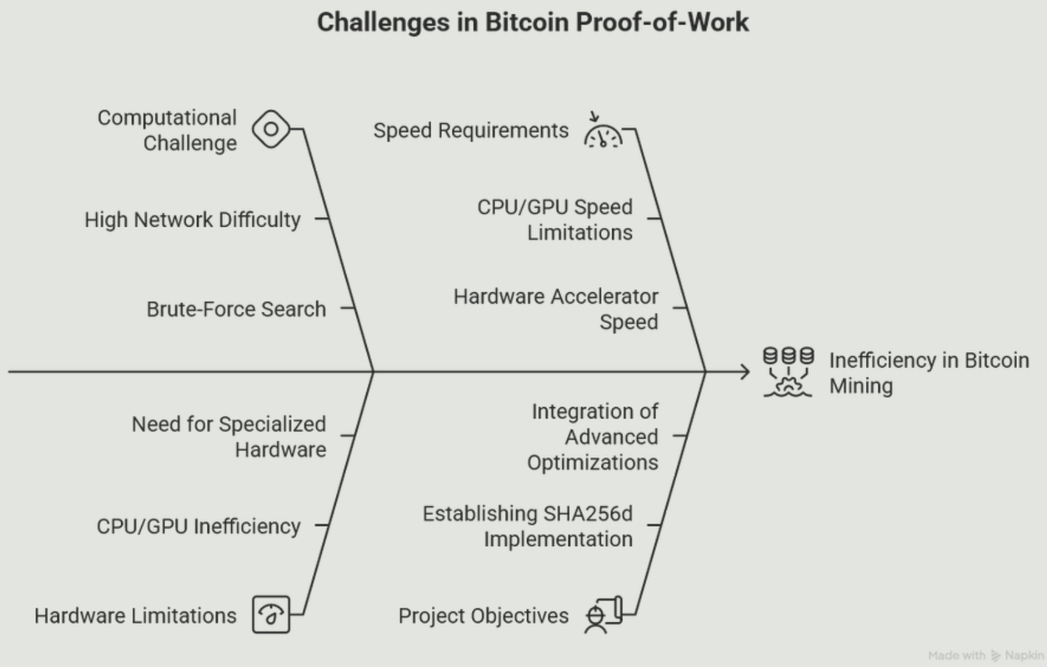
INTRODUCTION

- **Bitcoin Proof-of-Work (PoW)**: Bitcoin's security mechanism requires miners to solve a cryptographic puzzle. This involves finding a 32-bit nonce that, when combined with a block header and subjected to SHA256d hashing, produces a hash value numerically below a network-defined target.
- **Typical Speed**: CPUs typically perform in the Kilohashes per second (KH/s) range, while GPUs achieve Megahashes per second (MH/s).
- **Hardware Accelerator Speed**: Dedicated hardware accelerators, such as FPGAs, can achieve Gigahashes per second (GH/s), and Application-Specific Integrated Circuits (ASICs) can reach Terahashes per second (TH/s) and even Petahashes per second (PH/s). This represents a 1,000x to 1,000,000x speed improvement over CPUs for this specific hashing task, demonstrating the critical need for specialized hardware.



CHALLENGES

- **Computational Challenge**: The current network difficulty demands an astronomical number of hash computations per second, ranging from billions to quadrillions, making brute-force nonce searching extremely demanding.
- **Inefficiency of CPUs/GPUs**: Traditional computing platforms are not optimized for the highly repetitive and specific bitwise operations of SHA256d.
- **Project Objective (Initial Phase)**: The primary goal of this initial phase is to establish a robust and accurate hardware implementation of the SHA256d function.
- **Future Foundation**: This foundational hardware will serve as the essential building block for subsequent phases, enabling the integration of advanced architectural optimizations like pipelining and massive parallelism.



WORKING PRINCIPLE

Key Hardware Principles Applied

Dedicated Hardware Implementation:

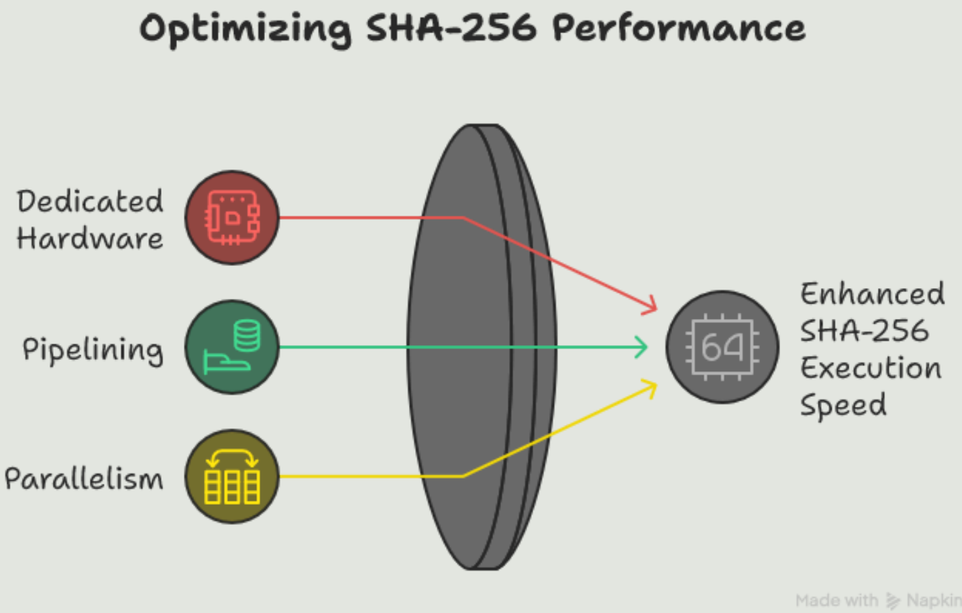
- SHA256d operations (bitwise logic, additions, rotations) are hardwired into custom digital circuits (logic gates, flip-flops).
- Eliminates instruction fetching/decoding overhead of CPUs/GPUs.
- Enables significantly faster execution for this specific cryptographic task.

Pipelining (within SHA-256 Core):

- The 64 rounds of the SHA-256 compression function are broken down into sequential stages.
- Registers separate these stages, allowing new data to enter the pipeline before previous data completes.
- Benefit: Maximizes throughput; after initial latency, one hash computation can begin almost every clock cycle. This is crucial for high-speed operation.

Parallelism (Architectural Foundation):

- While the initial phase focuses on a single functional SHA256d unit, the design is structured to readily support multiple identical hashing units.
- Future Extension: In subsequent phases, these parallel units will simultaneously process different nonces, linearly scaling the overall hash rate. This principle is inherent in the modular design of the SHA256d core and its wrapper.



WORKING

Bitcoin Block Header & Nonce

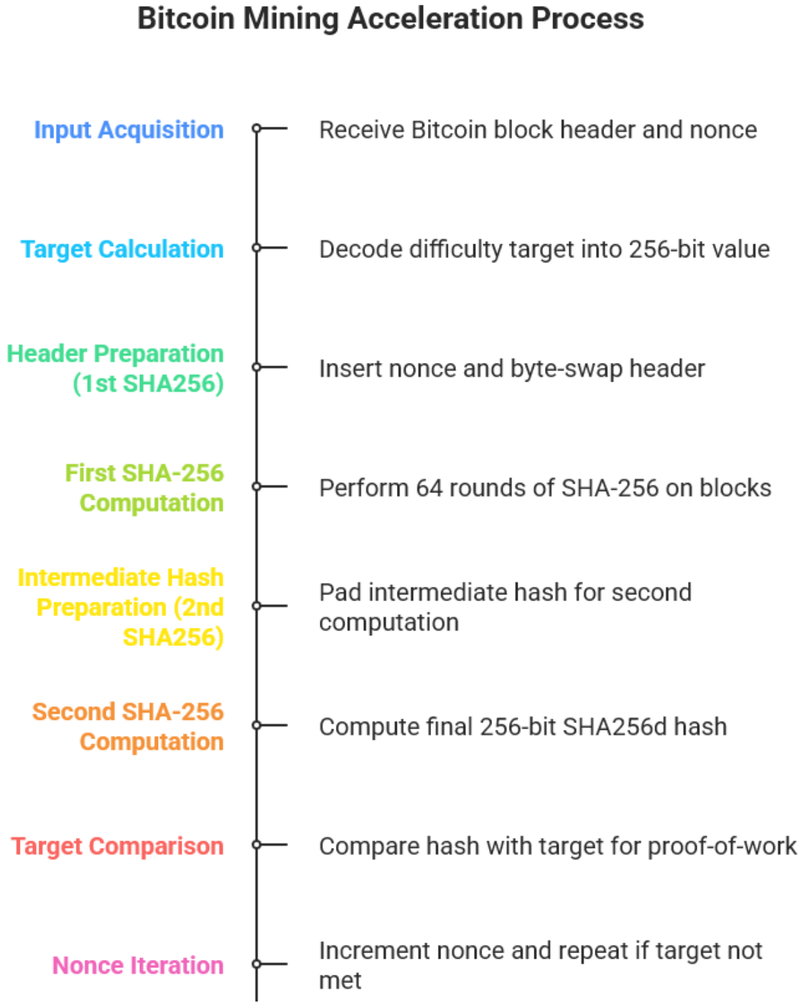
- Block Header: An 80-byte data structure containing critical metadata for a Bitcoin block.
- Nonce: A 32-bit (4-byte) field within the block header, systematically varied by miners.
- Other Fields: Block version, previous block hash, Merkle root, timestamp, and 'Bits' field (difficulty target).

SHA-256 Algorithm Overview

- Function: SHA-256 is a cryptographic hash function; arbitrary input → fixed 256-bit hash.
- Operation: Processes 512-bit (64-byte) message blocks; 64 iterative rounds of bitwise logic, modular additions, and cyclic rotations on eight 32-bit working variables, combined with message schedule words and round constants.

SHA256d (Double SHA-256)

- Bitcoin's Proof-of-Work: The Bitcoin protocol requires SHA-256 to be applied twice sequentially to the block header: Hfinal =SHA256(SHA256(Block Header)).



RESULTS

Functional Verication (Simulation):

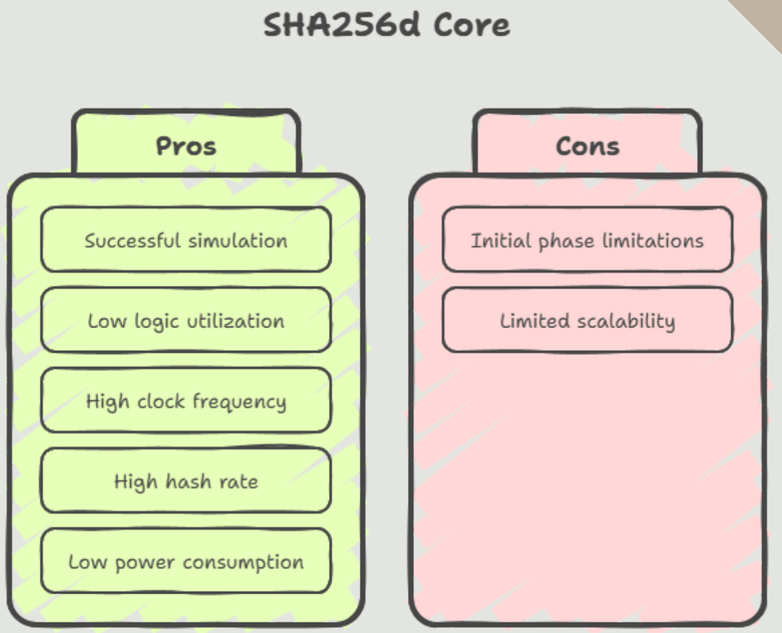
◯ Successfully simulated the SHA256d computation unit.
◯ Veried correct hash outputs for known Bitcoin block header test vectors.
◯ Conrmed accurate nonce iteration and target comparison logic in simulation.

Synthesis Results (Expected for Initial Phase):

◯ **Logic Utilization:** Low to moderate utilization of FPGA Look-Up Tables (LUTs) and Flip-Flops (FFs) for a single SHA256d core.
◯ **Estimated Clock Frequency (Fmax):** Expected to achieve clock frequencies in the hundreds of MHz (e.g., 200-300 MHz) for a pipelined SHA-256 core on modern FPGAs.

Performance Metrics (Projected for Initial Phase):

◯ **Hash Rate (Single Core):** A single, pipelined SHA256d core operating at 250 MHz could theoretically achieve approximately 250 Megahashes per second (MH/s).
◯ **Power Consumption**: Signicantly lower power consumption per hash compared to CPU/GPU implementations for the core hashing logic.
◯ **Efficiency:** Demonstrates the potential for superior energy eciency (J/GH) compared to general-purpose hardware.



Jun 3 09:30

mini_bitcoin - [/home/mannansaood/mini_bitcoin/mini_bitcoin.xpr] - Vivado 2024.2

File Edit Flow Tools Reports Window Layout View Help Quick Access

Synthesis Complete

Default Layout

Flow Navigator

- PROJECT MANAGER
  - Settings
  - Add Sources
  - Language Templates
  - IP Catalog
- IP INTEGRATOR
  - Create Block Design
  - Open Block Design
  - Generate Block Design
- SIMULATION
  - Run Simulation
- RTL ANALYSIS
  - Run Linter
  - Open Elaborated Design
    - Report Methodology
    - Report DRC
    - Report Noise
    - Schematic
- SYNTHESIS
  - Run Synthesis
  - Open Synthesized Design
    - Constraints Wizard
    - Edit Timing Constraints
    - Set Up Debug
    - Open Dataflow Design
    - Report Timing Summary
    - Report Clock Networks
    - Report Clock Interaction

ELABORATED DESIGN - xc7a35tcpg236-1

Sources | Netlist

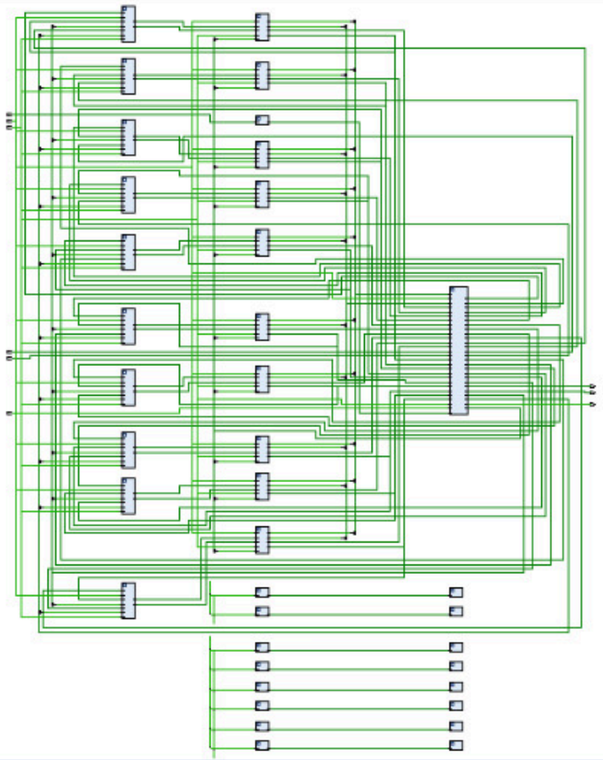
- bitcoin_miner_accel_top
  - Nets (23640)
  - Leaf Cells (2)
  - 33-23.25sha256_pkg::H_INIT[0]_inferred (keep_16)
  - 33-23.25sha256_pkg::H_INIT[0]_inferred_0 (keep_17)
  - 33-23.25sha256_pkg::H_INIT[1]_inferred (keep_18)
  - 33-23.25sha256_pkg::H_INIT[1]_inferred_0 (keep_19)
  - 33-23.25sha256_pkg::H_INIT[2]_inferred (keep_20)
  - 33-23.25sha256_pkg::H_INIT[2]_inferred_0 (keep_21)
  - 33-23.25sha256_pkg::H_INIT[3]_inferred (keep_22)
  - 33-23.25sha256_pkg::H_INIT[3]_inferred_0 (keep_23)
  - 33-23.25sha256_pkg::H_INIT[4]_inferred (keep_24)
  - 33-23.25sha256_pkg::H_INIT[4]_inferred_0 (keep_25)

Tile Properties

Select an object to see properties

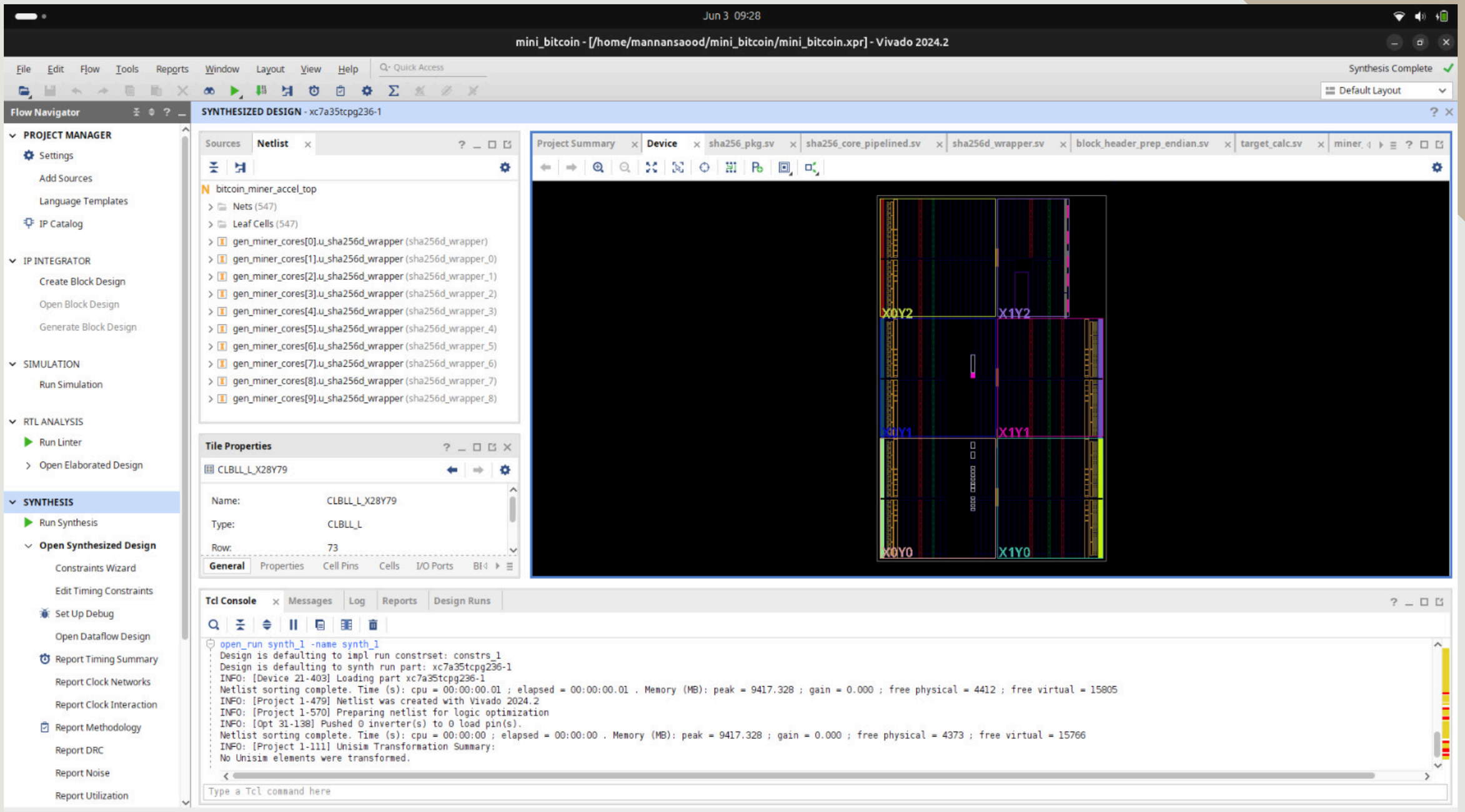
Project Summary | Schematic | sha256_pkg.sv | sha256_core_pipelined.sv | sha256d_wrapper.sv | block_header_prep_endian.sv | target_calc.sv | mi

38 Cells 996 I/O Ports 23640 Nets

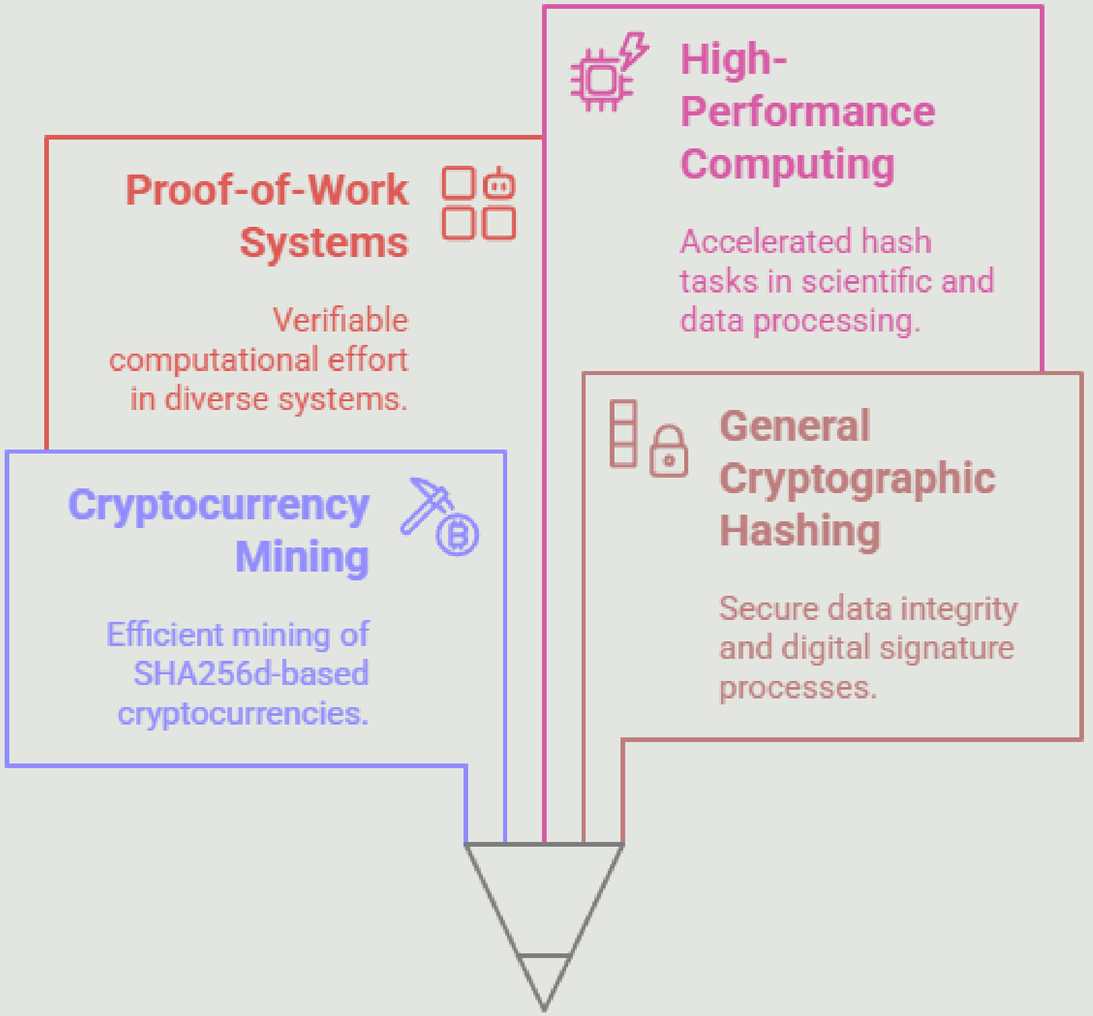


Tcl Console | Messages | Log | Reports | **Design Runs**

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy
synth_1	constrs_1	synth_design Complete!												256	0	0	0	0	6/3/25, 9:26 AM	00:01:00	Vivado Synthesis Defaults (Vivad
impl_1	constrs_1	Not started																			Vivado Implementation Defaults


Jun 3 09:28
mini_bitcoin - [/home/mannansaood/mini_bitcoin/mini_bitcoin.xpr] - Vivado 2024.2
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Synthesis Complete
Default Layout
Flow Navigator
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Run Linter
Open Elaborated Design
SYNTHESIS
Run Synthesis
Open Synthesized Design
Constraints Wizard
Edit Timing Constraints
Set Up Debug
Open Dataflow Design
Report Timing Summary
Report Clock Networks
Report Clock Interaction
Report Methodology
Report DRC
Report Noise
Report Utilization
SYNTHESIZED DESIGN - xc7a35tcpg236-1
Sources
Netlist
bitcoin_miner_accel_top
Nets (547)
Leaf Cells (547)
gen_miner_cores[0].u_sha256d_wrapper (sha256d_wrapper)
gen_miner_cores[1].u_sha256d_wrapper (sha256d_wrapper_0)
gen_miner_cores[2].u_sha256d_wrapper (sha256d_wrapper_1)
gen_miner_cores[3].u_sha256d_wrapper (sha256d_wrapper_2)
gen_miner_cores[4].u_sha256d_wrapper (sha256d_wrapper_3)
gen_miner_cores[5].u_sha256d_wrapper (sha256d_wrapper_4)
gen_miner_cores[6].u_sha256d_wrapper (sha256d_wrapper_5)
gen_miner_cores[7].u_sha256d_wrapper (sha256d_wrapper_6)
gen_miner_cores[8].u_sha256d_wrapper (sha256d_wrapper_7)
gen_miner_cores[9].u_sha256d_wrapper (sha256d_wrapper_8)
Tile Properties
CLBLL_L_X28Y79
Name: CLBLL_L_X28Y79
Type: CLBLL_L
Row: 73
General Properties Cell Pins Cells I/O Ports
Project Summary
Device
sha256_pkg.sv
sha256_core_pipelined.sv
sha256d_wrapper.sv
block_header_prep_endian.sv
target_calc.sv
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0
Tcl Console
Messages
Log
Reports
Design Runs
open_run synth_1 -name synth_1
Design is defaulting to impl run constrset: constrs_1
Design is defaulting to synth run part: xc7a35tcpg236-1
INFO: [Device 21-403] Loading part xc7a35tcpg236-1
Netlist sorting complete. Time (s): cpu = 00:00:00.01 ; elapsed = 00:00:00.01 . Memory (MB): peak = 9417.328 ; gain = 0.000 ; free physical = 4412 ; free virtual = 15805
INFO: [Project 1-479] Netlist was created with Vivado 2024.2
INFO: [Project 1-570] Preparing netlist for logic optimization
INFO: [Opt 31-138] Pushed 0 inverter(s) to 0 load pin(s).
Netlist sorting complete. Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 9417.328 ; gain = 0.000 ; free physical = 4373 ; free virtual = 15766
INFO: [Project 1-111] Unisim Transformation Summary:
No Unisim elements were transformed.
Type a Tcl command here

APPLICATIONS



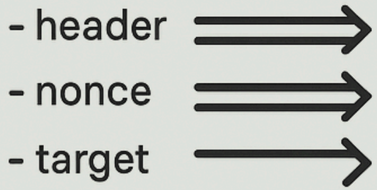
FUTURE WORK/FUTURE SCOPE

Deep Pipelining

- Further optimize the SHA-256 core pipeline depth to achieve higher clock frequuencis and maximize throughput per individual hashing unit

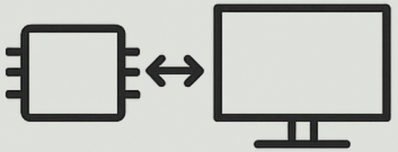
Massive Parallelism

- Instantiate numerous independent SHA256d units on the FPGA
- Implement efficient nonce distribution and result coleition mechanisms for hundrods or thousands of parallel cores
- Dramatically increase the oveall hash rate (e.g., from GH/s to TH/s)



Optimized Data Flow & Control

- Refine the data paths for block header preparation, nonce iteration, aml target comparison to minimize latency and maximize end-to-end efficiency



Optimized Data Flow & Control

- Refine the data paths for block header preparation, nonce iteration, and target comparison to minimize latency and maximize end-to-end efficiency

Host Interface Integration

- Implement a high-bandwidth interface (e.g. PCle, AXII for seamless communication with a host CPU, allowing for efficient task loading and result reporting

CONCLUSION

- **Current Achievement:**

We've successfully built and tested a basic hardware version of the SHA256d algorithm — the core of Bitcoin mining.

- **Key Advantage:**

The hardware design offers much better speed and energy efficiency compared to software-based miners.

- **Validation:**

Core mining functions like hashing, data processing, and target checking work correctly in this setup.

- **Next Steps:**

1. **Deep Pipelining:** Enable each SHA-256 unit to start a new hash every clock cycle for maximum throughput.
2. **Massive Parallelism:** Deploy hundreds or thousands of SHA256d units to test more nonces in parallel, greatly boosting hash rate.
3. **Optimized Data Flow:** Improve how block headers, nonces, and targets are handled to reduce delays and increase system efficiency.

REFERNCE WORK

High-Throughput Pipelined SHA-256 Cores:

- M. A. A. Al-Rjoub & M. A. Al-Qaralleh (2014): Their work, "High-Throughput SHA-256 Hash Function Implementation on FPGA," details the design and synthesis of deeply pipelined SHA-256 compression functions.
- H. Al-Khateeb et al. (2012): In "Efficient FPGA Implementation of SHA-256 Hash Function," they explore various architectural optimizations for the SHA-256 core, such as loop unrolling and parallel execution units.

Multi-Core FPGA Mining Architectures:

- L. Ma et al. (2014): Their paper, "FPGA-based Bitcoin Miner Design," discusses the integration of multiple SHA-256d cores onto a single FPGA, along with sophisticated control logic for nonce distribution and result aggregation.
- S. A. Al-Mawali & A. Al-Hinai (2015): In "Design and Implementation of a High-Performance Bitcoin Mining Hardware on FPGA," they present an FPGA-based miner architecture specifically designed for parallelism.

Thank You